

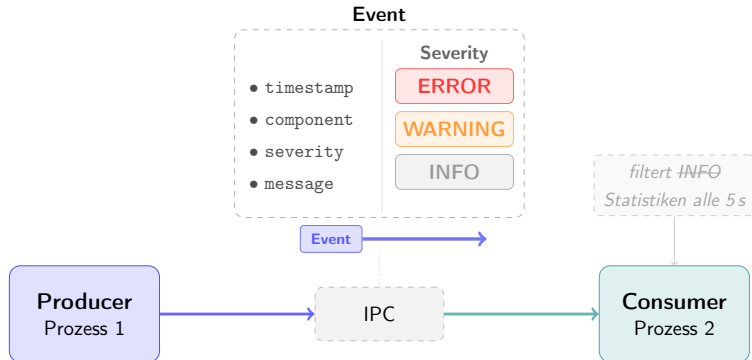
Modulares Producer-Consumer-System mit austauschbarer IPC

Bewerbung bei SEW-EURODRIVE, Arbeitsprobe

Jonas Dressner

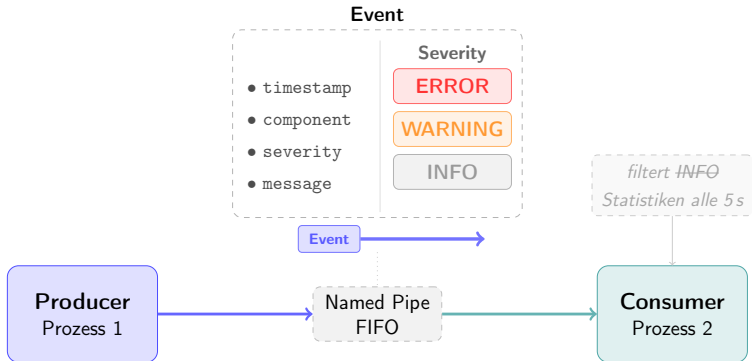
02.06.2026

- 1 Anforderungen
- 2 Design und Architektur
- 3 Zusammenfassung



- **Producer und Consumer** laufen in getrennten Prozessen und kommunizieren via IPC
- **IPC-Implementierung** muss austauschbar sein (Named Pipes, Sockets, Shared Memory, ...)
- IPC mit **Systemfunktionen** (Windows oder Linux)

Design: IPC via Named Pipe / FIFO



- **Producer = Pipe-Server, Consumer = Pipe-Client:** Producer legt die Pipe an, Consumer verbindet sich
- Windows: **Named Pipes** (CreateNamedPipeA, WriteFile/ReadFile)
- Linux: **FIFOs** (mkfifo, open, poll)

Komponentendiagramm

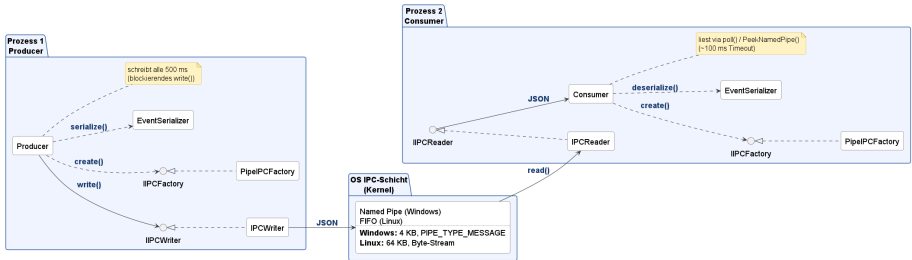


Abbildung 1: Komponentendiagramm: Zwei Prozesse, verbunden über die OS-IPC-Schicht.

- **Synchrone IPC:** blockierendes `write()`, Consumer liest gepuffert mit `poll()/Timeout`
- **Symmetrische Architektur:** gleiches Binary für beide Prozesse, Rolle via CLI-Argument (`-producer` / `-consumer`)

Architekturdiagramm

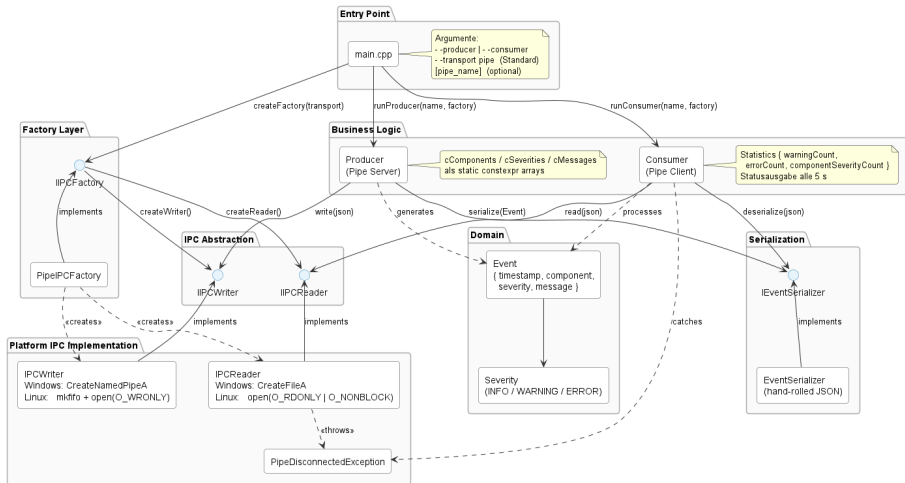


Abbildung 2: Schichten, Interfaces und Abhängigkeiten.

Architektur

- **Abstract Factory:** Transport austauschbar (IIPCFactory)
- **Interface Segregation:** möglichst schlanke Interfaces
- **Dependency Injection** im Konstruktor → testbar mit Fakes
- **Symmetrisches Binary:** Rolle via CLI-Flag

Design

- **Producer = Pipe-Server, Consumer = Pipe-Client**
- **Synchrone IPC:** blockierendes `write()`, gepufferte `poll()/PeekNamedPipe`
- **Robustheit:** typisierte `PipeDisconnectedException` + Reconnect-Logik
- **C++17:** RAII, Smart Pointer, `std::atomic`, `constexpr`

Vielen Dank!

Fragen?

Source Code & Dokumentation:

github.com/JonasDressner/event_system

Vollständiges Klassendiagramm

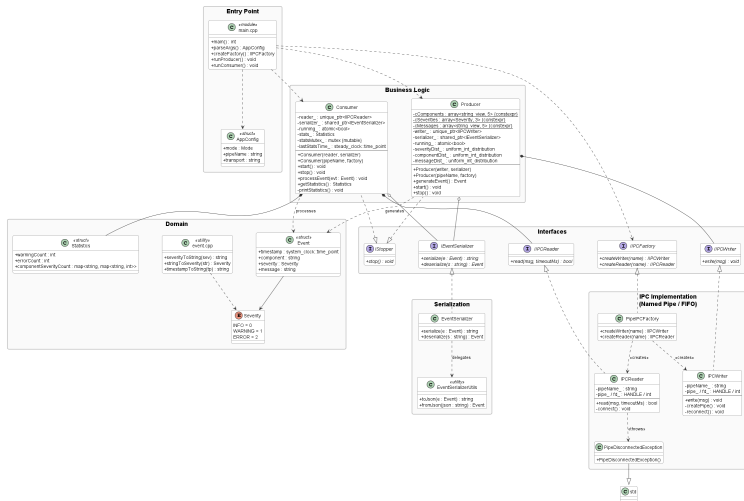


Abbildung 3: Vollständiges Klassendiagramm: Alle Klassen, Member und Beziehungen.

Interface

```
struct IIPCFactory {  
    virtual ~IIPCFactory() = default;  
    virtual unique_ptr<IIPCWriter>  
        createWriter(string name) = 0;  
    virtual unique_ptr<IIPCReader>  
        createReader(string name) = 0;  
};  
  
// main.cpp  
if (transport == "pipe")  
    return make_unique<PipeIPCFactory>();  
// transport == "tcp" --> erweiterbar
```

Plattform-Implementierung

Windows (Named Pipes):

- CreateNamedPipeA
- ConnectNamedPipe
- WriteFile / ReadFile
- PeekNamedPipe (Timeout)

Linux (FIFOs):

- mkfifo
- open(O_WRONLY)
- open(O_RDONLY|O_NONBLOCK)
- poll() (Timeout)